

Reinforcement Learning

COMPLETE TUTORIAL: MAB, Monte Carlo, TD, DQN, Policy Methods

GARP RAI Exam Preparation

What You Will Learn in This Tutorial:

Topic	Key Concepts Covered
1. RL Fundamentals	Agent, Environment, State, Action, Reward, Policy, Value Functions
2. Multi-Armed Bandit	Exploration vs Exploitation, ϵ -Greedy, Q-Value Updates
3. Monte Carlo Methods	Episode-based Learning, Full Returns, Bias-Variance Trade-off
4. Temporal Difference	Online Learning, Bootstrapping, Q-Learning, SARSA, TD Error
5. Deep RL & DQN	Neural Networks, Experience Replay, Target Networks
6. Policy Methods	Value-based vs Policy-based, Actor-Critic, Continuous Actions
7. MDP & Bellman	Markov Property, Dynamic Programming, Optimality Equations
8. Advanced Topics	RLHF, AlphaGo, Model-Based RL, Continuous RL

Key to Success: Understand the "exploration vs exploitation" trade-off!

RL - What Makes It Different?

Reinforcement Learning vs Other Paradigms

Paradigm	Key Characteristic
Supervised Learning	Learns from labeled data ; output = prediction/classification
Unsupervised Learning	Finds hidden patterns ; output = clusters/structures
Reinforcement Learning	Learns through trial and error ; output = policy/action

Core RL Concept:

- Agent **interacts** with environment
- Takes **actions**
- Receives **rewards** (positive or negative)
- Goal: **Maximize cumulative future reward**

Analogy - Dog Training:

- Command → Dog takes action → Reward if correct
- Over time, dog learns the **policy**

Exam Trap: RL uses **rewards**, not labels!

The Language of Reinforcement Learning

Term	Definition
Agent	The learner/decision-maker who takes actions
Environment	The world the agent interacts with
Action (a)	A choice the agent can make
State (s)	The current situation of the environment
Reward (R)	Feedback from environment (positive or negative)
Policy (π)	The agent's strategy - maps states to actions
Value Function $V(s)$	How good it is to be in state s
Action-Value $Q(s,a)$	How good it is to take action a in state s

Key Insight:

- Optimal policy: $\pi^*(s) = \arg \max_a Q(s, a)$

Exam Trap: Policy $\neq$ Value Function!

The Mathematics of RL

Expected Future Return (G):

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

$\gamma = \text{discount factor}$ ($0 \leq \gamma \leq 1$)

State-Value Function $V(s)$:

$$V_{\pi}(s) = E_{\pi}[G_t | S_t = s]$$

Action-Value Function $Q(s,a)$:

$$Q_{\pi}(s, a) = E_{\pi}[G_t | S_t = s, A_t = a]$$

Relationship:

$$V_{\pi}(s) = \sum_a \pi(a|s) Q_{\pi}(s, a)$$

$$V^*(s) = \max_a Q^*(s, a)$$

Exam Trap: $V(s)$ = expectation over actions; $Q(s,a)$ = action-specific!

Real-World RL Applications

AlphaGo / AlphaZero (DeepMind):

- Superhuman performance in Go, Chess, Shogi
- Played [millions](#) of games against itself
- Discovered [novel strategies](#)

Robotics:

- Complex manipulation tasks
- Self-driving cars
- Warehouse automation

Business:

- Dynamic Pricing
- Supply Chain Optimization
- Algorithmic Trading
- [RLHF for ChatGPT](#) (Reinforcement Learning from Human Feedback)

Exam Trap: RLHF is key for LLM fine-tuning!

The Multi-Armed Bandit Problem

The Scenario:

- Gambler faces **multiple slot machines**
- Each machine has **unknown** reward distribution
- Goal: **Maximize total winnings**

Key MAB Characteristics:

- 1 **Single state** - environment doesn't change
- 2 Actions don't affect future states
- 3 Each action gives **immediate reward**
- 4 No long-term consequences

The Core Dilemma:

- **Exploit**: Play best machine so far
- **Explore**: Try other machines to find better ones

Exam Trap: MAB has **ONLY ONE STATE!**

Three Strategies for MAB

Strategy 1: Greedy (Pure Exploitation)

- Always choose **highest Q-value**
- Pros: Maximizes immediate reward
- Cons: May miss better machine

Strategy 2: Random (Pure Exploration)

- Always choose **random** machine
- Pros: Gathers information about all machines
- Cons: Never uses knowledge

Strategy 3: ϵ -Greedy (Balanced)

- With probability ϵ : **Explore** (random)
- With probability $1 - \epsilon$: **Exploit** (best)
- **Balances** exploration and exploitation

Exam Trap: ϵ -greedy with decay is **best practical**!

Q-Value = Average of Observed Rewards

$$Q_k^n = \frac{1}{n} \sum_{j=1}^n R_j$$

Incremental Update:

$$Q_k^n = Q_k^{n-1} + \frac{1}{n}(R_n - Q_k^{n-1})$$

Example: Machine A played 5 times:

- Rewards: [1.2, 0.8, 1.5, 0.3, 1.1]
- $Q(A) = (1.2+0.8+1.5+0.3+1.1)/5 = 0.98$

Step-by-Step:

- 1 Play 1: $R=1.2$, $Q=1.2$
- 2 Play 2: $R=0.8$, $Q=(1.2+0.8)/2=1.0$
- 3 Play 3: $R=1.5$, $Q=(1.2+0.8+1.5)/3=1.167$

Exam Trap: Q-values are **averages**, not sums!

How ϵ Decays Over Time

$$\epsilon_t = \beta^{t-1}$$

where β = decay factor ($0 < \beta < 1$)

Example with $\beta = 0.85$:

Trial	ϵ_t	Decision
1	1.00	Always Explore
2	0.85	85% Explore, 15% Exploit
5	0.522	52.2% Explore
10	0.232	23.2% Explore
20	0.046	4.6% Explore
50	0.0004	Pure Exploit

Key Insight:

- High ϵ early = More exploration
- Low ϵ later = More exploitation

Exam Trap: $\epsilon_t = \beta^{t-1}$, not β^t !

MAB - Step-by-Step Example

Setup:

- 3 machines: A, B, C
- True means: $\mu_A = 0.8$, $\mu_B = 0.5$, $\mu_C = 0.7$
- $\beta = 0.85$

Trial 1: $\epsilon = 1.00$, random=0.7 $\rightarrow$ Explore

- Choose A, Reward=1.2, $Q(A)=1.2$

Trial 2: $\epsilon = 0.85$, random=0.99 $\rightarrow$ Exploit

- Choose A (best), Reward=0.8, $Q(A)=1.0$

Trial 3: $\epsilon = 0.7225$, random=0.65 $\rightarrow$ Explore

- Choose B, Reward=1.7, $Q(B)=1.7$

Machine	Q-Value
A	0.80
B	0.50
C	0.70

After 10 Trials:

Exam Trap: Estimates may differ from true means due to sampling error!

The Risk of Pure Exploitation

Scenario:

- $Q(A) = 1.5$ (lucky), $Q(B) = 0$, $Q(C) = 0$
- True means: $\mu_A = 0.2$, $\mu_B = 2.0$, $\mu_C = 1.8$

Greedy Strategy:

- Always plays A ($Q=1.5$)
- Expected reward = 0.2 per play
- Total reward = $0.2 \times 1000 = 200$

Optimal Strategy:

- Always plays B (true mean = 2.0)
- Total reward = $2.0 \times 1000 = 2000$

Regret:

- Regret = $2000 - 200 = 1800$ (huge loss!)

Exam Trap: Greedy can be catastrophically suboptimal!

MAB - Random Strategy Problem

The Problem of Pure Exploration

Scenario:

- 3 machines: $\mu_A = 0.8$, $\mu_B = 0.5$, $\mu_C = 0.7$
- 1000 plays

Random Strategy:

- Expected reward = 0.667 per play
- Total = $0.667 \times 1000 = 667$

Optimal:

- Always plays A (0.8)
- Total = $0.8 \times 1000 = 800$

Comparison:

Strategy	Expected Reward
Optimal	800
ϵ -greedy (0.1)	787
Greedy (if lucky)	800
Greedy (if unlucky)	200
Random	667

Exam Trap: Random **never exploits** - wastes knowledge!

ϵ -Greedy - The Balanced Approach

Setup:

- $\epsilon = 0.10$ (constant)
- 1000 plays

Expected Behavior:

- 90% plays: best machine (A) $\rightarrow$ reward 0.8
- 10% plays: random $\rightarrow$ average 0.667
- Expected reward $= 0.9(0.8) + 0.1(0.667) = 0.787$
- Total ≈ 787

With Decay:

- Early: More exploration
- Late: More exploitation
- Often **outperforms** constant ϵ

Exam Trap: ϵ -greedy with decay is **usually** better!

Choosing the Decay Factor β

Effect of Different β :

Trial	$\beta = 0.85$	$\beta = 0.95$	$\beta = 0.99$
1	1.00	1.00	1.00
5	0.522	0.815	0.961
10	0.232	0.630	0.914
20	0.046	0.377	0.827
50	0.0004	0.077	0.606
100	≈ 0	0.006	0.366

Interpretation:

- $\beta = 0.85$: Fast decay, quick exploitation
- $\beta = 0.95$: Moderate decay, balanced
- $\beta = 0.99$: Slow decay, long exploration

Exam Trap: Smaller β = faster decay = less exploration later!

Thompson Sampling - A Bayesian Alternative

The Concept:

- Maintain **probability distribution** for each machine
- Sample from each distribution
- Choose **maximum sampled value**
- Naturally balances exploration and exploitation

Example - Binary Rewards:

- A: Beta(5,2) $\rightarrow$ mean=0.714
- B: Beta(2,3) $\rightarrow$ mean=0.400
- C: Beta(3,2) $\rightarrow$ mean=0.600
- Sample: A=0.72, B=0.35, C=0.68 $\rightarrow$ Choose A

Advantages:

- No ϵ parameter to tune
- Balances based on uncertainty
- Theoretically optimal (Bayesian)

Exam Trap: Thompson Sampling is **Bayesian!**

Regret - Measuring Performance

$$\text{Regret}(T) = T \times \mu^* - \sum_{t=1}^T \mu_{a_t}$$

Interpretation:

- Opportunity cost of not knowing the best machine
- Cumulative loss from suboptimal choices
- Goal: **Minimize regret**

Example:

- $\mu^* = 0.8$ (Machine A)
- Play B 100 times $\rightarrow$ regret = $100(0.8-0.5) = 30$

Expected Regret (T=1000):

Strategy	Expected Regret
Optimal	0
ϵ -greedy (0.1)	13
Greedy (unlucky)	600
Random	133

Exam Trap: Regret is cumulative loss - minimize it!

A/B Testing as a Multi-Armed Bandit

Traditional A/B Testing:

- Split traffic 50/50
- Test for significance
- Problem: **Wastes traffic** on inferior options

MAB Approach:

- Each variant = **slot machine**
- Use ϵ -greedy or Thompson Sampling
- Dynamically allocate more traffic to better variants

Example - Ad CTR:

- Variant A: 5% CTR, B: 3%, C: 4%

Traditional (100,000 impressions):

- Expected clicks = $0.05(33,333) + 0.03(33,333) + 0.04(33,333) = 4,000$

MAB Approach:

- Early: Explore all variants
- Later: 90% to best (A)
- Expected clicks $\approx 4,850$
- **21% more clicks!**

Exam Trap: MAB significantly outperforms A/B testing!

MAB - Key Takeaways

Key Concepts:

1 MAB Characteristics:

- Single state, immediate rewards, no future consequences

2 Three Strategies:

- Greedy = Pure exploitation
- Random = Pure exploration
- ϵ -Greedy = Balanced (best)

3 Q-Value:

$$Q_k^n = \frac{1}{n} \sum R_j$$

4 ϵ Decay:

$$\epsilon_t = \beta^{t-1}$$

5 Regret:

- Loss from not knowing the optimal action

Exam Success: MAB is the foundation - master it!

Monte Carlo - Learning from Complete Episodes

What is Monte Carlo RL?

- Learns from **complete episodes**
- Episode = full sequence from start to terminal state
- Example: Complete game of chess

Key Characteristics:

- **Wait until episode ends** to update
- Uses **actual returns** (G_t)
- **Unbiased** - learns from real experience

Update Formula:

$$Q(s, a) \leftarrow Q(s, a) + \alpha(G_t - Q(s, a))$$

Exam Trap: MC requires **complete episodes**!

What is an Episode?

Definition:

- Complete trajectory from start to terminal state
- Sequence: $S_0, A_0, R_1, S_1, A_1, \dots, S_T$

Examples:

- 1 Nim: Start $\rightarrow$ coin removal $\rightarrow$ win/lose
- 2 Chess: Opening $\rightarrow$ moves $\rightarrow$ Checkmate
- 3 Board Game: Start $\rightarrow$ moves $\rightarrow$ End

Return Calculation:

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-t-1} R_T$$

Update Only at Episode End:

- No updates within episode
- All state-action pairs updated at end

Exam Trap: MC updates **only at episode end!**

Calculating Returns in MC

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}$$

Example - Nim with $\gamma = 1$:

- Episode: $S=6 \rightarrow S=3 \rightarrow S=1 \rightarrow S=0$ (Win)
- Rewards: $R_1 = 0, R_2 = 0, R_3 = 1$
- $G_0 = 0 + 0 + 1 = 1$
- $G_1 = 0 + 1 = 1$
- $G_2 = 1$

Example - Nim with $\gamma = 0.9$:

- $G_0 = 0 + 0 + 0.81 = 0.81$
- $G_1 = 0 + 0.9 = 0.9$
- $G_2 = 1$

Why Discount?

- Future rewards less certain
- More weight to immediate rewards

Exam Trap: MC uses actual return G_t !

Monte Carlo - First-Visit vs Every-Visit

First-Visit vs Every-Visit MC

The Distinction:

- Same state-action pair may be visited multiple times

First-Visit MC:

- Only update **first time** (s, a) occurs
- **Unbiased** estimate

Every-Visit MC:

- Update **every time** (s, a) occurs
- **Slightly biased** but lower variance

Example:

- Episode: $(S1, A1), (S2, A2), (S1, A1), \text{Terminal}$
- First-visit: Update $(S1, A1)$ once
- Every-visit: Update $(S1, A1)$ twice

Exam Trap: First-visit = unbiased; Every-visit = lower variance!

MC NIM - First Episode

Setup:

- 6 coins, remove 1, 2, or 3
- $\alpha = 0.05$, $\gamma = 1$
- Opponent plays randomly

Episode 1:

- 1 $S=6$, $A=3 \rightarrow S'=3$ (opponent picks 2)
- 2 $S=1$, $A=1 \rightarrow S'=0$ (Player 1 wins!)
- 3 Return: $G = 1$

MC Updates (Episode End):

- $Q(6,3) = 0 + 0.05(1-0) = 0.05$
- $Q(1,1) = 0 + 0.05(1-0) = 0.05$

After Episode 1 Q-Table:

Action	S=1	S=2	S=3	S=4	S=5	S=6
Pick 1	0.05	0	0	0	0	0
Pick 2	0	0	0	0	0	0
Pick 3	0	0	0	0	0	0.05

Exam Trap: MC updates all pairs in episode with same G!

MC NIM - Second Episode

Episode 2:

- 1 $S=6, A=1 \rightarrow S'=5$ (opponent picks 1)
- 2 $S=4, A=3 \rightarrow S'=1$ (opponent picks 1)
- 3 $S=0$ (Player 1 loses!)
- 4 Return: $G = -1$

MC Updates:

- $Q(6,1) = 0 + 0.05(-1-0) = -0.05$
- $Q(4,3) = 0 + 0.05(-1-0) = -0.05$

After Episode 2:

Action	S=1	S=2	S=3	S=4	S=5	S=6
Pick 1	0.05	0	0	0	0	-0.05
Pick 2	0	0	0	0	0	0
Pick 3	0	0	0	-0.05	0	0.05

Exam Trap: Negative G updates Q-values downward!

MC NIM - Third Episode

Episode 3:

- 1 $S=6, A=3 \rightarrow S'=3$ (opponent picks 2)
- 2 $S=1, A=1 \rightarrow S'=0$ (Player 1 wins!)
- 3 Return: $G = 1$

MC Updates:

- $Q(6,3) = 0.05 + 0.05(1-0.05) = 0.0975$
- $Q(1,1) = 0.05 + 0.05(1-0.05) = 0.0975$

After Episode 3:

Action	S=1	S=2	S=3	S=4	S=5	S=6
Pick 1	0.0975	0	0	0	0	-0.05
Pick 2	0	0	0	0	0	0
Pick 3	0	0	0	-0.05	0	0.0975

Exam Trap: Q-values gradually move toward true values!

MC NIM - After 1,000,000 Episodes

Final Q-Table:

Action	S=1	S=2	S=3	S=4	S=5	S=6
Pick 1	1	-1	-0.071	0.380	0	0.774
Pick 2	0	1	-1	-0.110	0	1
Pick 3	0	0	1	-0.858	0	0.020

Optimal Strategy:

- S=6: Pick 2 (Q=1)
- S=4: Pick 1 (Q=0.380)
- S=3: Pick 3 (Q=1)
- S=2: Pick 2 (Q=1)
- S=1: Pick 1 (Q=1)

Exam Trap: Q=1 = winning action; Q=-1 = losing action!

MC - Pros and Cons

Advantages:

- 1 Unbiased estimates - uses actual returns
- 2 Simple to understand
- 3 Works well with long episodes
- 4 No bootstrapping

Disadvantages:

- 1 High variance
- 2 Requires complete episodes
- 3 Slow learning (wait until episode end)
- 4 Inefficient for long episodes

Comparison:

MC	TD
Unbiased	Biased
High variance	Low variance
Episode-end	Step-by-step
Episodic only	Episodic + Continuous

Exam Trap: MC = unbiased but high variance!

Monte Carlo - Key Takeaways

Key Concepts:

1 Definition:

- Learns from complete episodes
- Updates only at episode end
- Uses actual returns (G_t)

2 Update Formula:

$$Q(s, a) \leftarrow Q(s, a) + \alpha(G_t - Q(s, a))$$

3 Return Calculation:

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots$$

4 First-Visit vs Every-Visit:

- First-visit: unbiased
- Every-visit: lower variance

5 Requirements:

- **Must have** terminal states
- Cannot handle continuous tasks

Exam Success: MC = episodic tasks only!

Temporal Difference - Learning Step by Step

What is TD Learning?

- Learns from **incomplete episodes**
- Updates **after each step**
- Uses **bootstrapping** - estimates of future values

Key Characteristic:

- Doesn't wait for episode end
- Updates using $R_{t+1} + \gamma V(s')$
- **Biased** but **lower variance**

TD Update:

$$V(s) \leftarrow V(s) + \alpha(R_{t+1} + \gamma V(s') - V(s))$$

TD Error:

$$\delta = R_{t+1} + \gamma V(s') - V(s)$$

= the "surprise"

Exam Trap: TD uses bootstrapping!

Bootstrapping - The Core of TD

What is Bootstrapping?

- Using a **current estimate** to update another estimate
- "Pulling yourself up by your own bootstraps"
- Instead of waiting for final outcome, use current best guess

MC vs TD Bootstrapping:

- **MC**: No bootstrapping - uses actual returns
- **TD**: Bootstrapping - uses estimates of future values

Example - Nim:

- $S=6$, action leads to $S'=1$
- MC: Wait until episode end to know G
- TD: Use $V(1)$ (current estimate) + immediate reward

TD Target:

$$R_{t+1} + \gamma V(s')$$

Exam Trap: Bootstrapping introduces bias but reduces variance!

Q-Learning - The Most Popular TD Algorithm

Update Formula:

$$Q(s, a) \leftarrow Q(s, a) + \alpha(R_{t+1} + \gamma \max_{a'} Q(s', a') - Q(s, a))$$

Key Features:

- ① **Off-policy:** Learns optimal policy regardless of behavior
- ② **Model-free:** No environment model needed
- ③ **TD method:** Uses bootstrapping

Why Off-Policy?

- Uses $\max_{a'} Q(s', a')$ (best possible action)
- Doesn't matter which action was actually taken
- Learns about **optimal policy**

Algorithm:

- ① Initialize $Q(s, a)$
- ② For each episode, for each step:
 - ① Choose a using ϵ -greedy
 - ② Take action, observe R, s'
 - ③ Update $Q(s, a)$
 - ④ $s \leftarrow s'$

Exam Trap: Q-learning is **off-policy**!

SARSA - On-Policy TD Learning

Update Formula:

$$Q(s, a) \leftarrow Q(s, a) + \alpha(R_{t+1} + \gamma Q(s', a') - Q(s, a))$$

where a' is the **actual action** taken in s'

Key Features:

- ① **On-policy**: Learns the policy being followed
- ② **Model-free**
- ③ **TD method**

SARSA vs Q-Learning:

Q-Learning	SARSA
Off-policy	On-policy
Uses $\max_{a'} Q(s', a')$	Uses $Q(s', a')$ (actual)
More aggressive	More conservative
Learns optimal	Learns current

Algorithm:

- ① Choose a using policy
- ② For each step:
 - ① Take action, observe R, s'
 - ② Choose a' from s' using policy
 - ③ Update $Q(s,a)$ using $Q(s',a')$

Exam Trap: SARSA is **on-policy**; Q-learning is **off-policy**!

Q-Learning vs SARSA - Detailed Comparison

Key Difference:

- Q-learning: $R + \gamma \max_{a'} Q(s', a')$
- SARSA: $R + \gamma Q(s', a')$

Behavior:

Q-Learning	SARSA
Best possible next action	Actual next action
More aggressive	More conservative
Learns risky policies	Learns safe policies

Example - Cliff Walking:

- Agent walks along cliff to reach goal
- Falling off cliff = large negative reward

Q-Learning:

- Learns optimal path (right next to cliff)
- Risky if agent makes mistakes

SARSA:

- Learns safer path (further from cliff)
- Accounts for suboptimal actions

Exam Trap: Q-learning = aggressive; SARSA = conservative!

TD NIM - First Episode

Setup:

- 6 coins, remove 1, 2, or 3
- $\alpha = 0.05$, $\gamma = 1$
- Opponent plays randomly

Episode 1:

- 1 $S=6$, $A=3 \rightarrow S'=3$, $R=0$
- 2 $S=1$, $A=1 \rightarrow S'=0$, $R=1$ (Win!)

TD Updates:

- **Step 1:** $Q(6,3) = 0 + 0.05(0 + 1(0) - 0) = 0$
- **Step 2:** $Q(1,1) = 0 + 0.05(1 + 1(0) - 0) = 0.05$

$$V(1) = 0.05$$

After Episode 1:

Action	S=1	S=2	S=3	S=4	S=5	S=6
Pick 1	0.05	0	0	0	0	0
Pick 2	0	0	0	0	0	0
Pick 3	0	0	0	0	0	0

Exam Trap: TD updates within episode using $V(s')$!

TD NIM - Second Episode

Episode 2:

- ① $S=6, A=1 \rightarrow S'=5, R=0$
- ② $S=4, A=3 \rightarrow S'=1, R=0$
- ③ $S=0$ (Player 1 loses!)

TD Updates:

- **Step 1:** $Q(6,1) = 0 + 0.05(0 + 1(0) - 0) = 0$
- **Step 2:** $Q(4,3) = 0 + 0.05(0 + 1(0.05) - 0) = 0.0025$

$V(4) = 0.0025$

After Episode 2:

Action	S=1	S=2	S=3	S=4	S=5	S=6
Pick 1	0.05	0	0	0	0	0
Pick 2	0	0	0	0	0	0
Pick 3	0	0	0	0.0025	0	0

Exam Trap: TD uses $V(1)$ from previous episode!

TD NIM - Third Episode

Episode 3:

- ① $S=6, A=3 \rightarrow S'=1, R=0$
- ② $S=1, A=1 \rightarrow S'=0, R=1$ (Win!)

TD Updates:

- **Step 1:** $Q(6,3) = 0 + 0.05(0 + 1(0.05) - 0) = 0.0025$
- **Step 2:** $Q(1,1) = 0.05 + 0.05(1 + 1(0) - 0.05) = 0.0975$

After Episode 3:

Action	S=1	S=2	S=3	S=4	S=5	S=6
Pick 1	0.0975	0	0	0	0	0
Pick 2	0	0	0	0	0	0
Pick 3	0	0	0	0.0025	0	0.0025

Exam Trap: TD values propagate backward through episode!

TD NIM - After 1,000,000 Episodes

Final Q-Table:

Action	S=1	S=2	S=3	S=4	S=5	S=6
Pick 1	1	-1	-0.105	0.383	0	0.845
Pick 2	0	1	-1	0.058	0	1
Pick 3	0	0	1	-0.842	0	0.011

Optimal Strategy:

- S=6: Pick 2
- S=4: Pick 1
- S=3: Pick 3
- S=2: Pick 2
- S=1: Pick 1

Key Insight:

- Both MC and TD converge to **same optimal strategy**
- TD learns faster than MC

Exam Trap: Both MC and TD converge to optimal policy!

TD - Pros and Cons

Advantages:

- 1 Can learn online - updates after each step
- 2 Works for continuous tasks
- 3 Lower variance than MC
- 4 More sample efficient
- 5 Often converges faster

Disadvantages:

- 1 Biased - uses estimates
- 2 May propagate errors
- 3 Requires careful tuning
- 4 Can be unstable

Comparison:

MC	TD
Unbiased	Biased
High variance	Low variance
Episode-end	Step-by-step
Episodic only	Episodic + Continuous

Exam Trap: TD = biased but lower variance!

TD(λ) - Bridging MC and TD

The Idea:

- MC = TD(1) - uses full return
- TD(0) = One-step TD
- TD(λ) - combines many n-step returns

n-Step Returns:

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V(s_{t+n})$$

λ -Return:

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$$

- $\lambda = 0$: TD(0)
- $\lambda = 1$: MC
- $0 < \lambda < 1$: Mix of n-step returns

Update:

$$Q(s, a) \leftarrow Q(s, a) + \alpha(G_t^\lambda - Q(s, a))$$

Exam Trap: $\lambda = 0 = \text{TD}$, $\lambda = 1 = \text{MC}$!

TD - Key Takeaways

Key Concepts:

1 TD Learning:

- Learns online, updates after each step
- Uses bootstrapping

2 TD Update:

$$V(s) \leftarrow V(s) + \alpha(R_{t+1} + \gamma V(s') - V(s))$$

3 TD Error:

$$\delta = R_{t+1} + \gamma V(s') - V(s)$$

4 Q-Learning:

- Off-policy, uses $\max_{a'} Q(s', a')$

5 SARSA:

- On-policy, uses $Q(s', a')$

6 TD(λ):

- $\lambda = 0 = \text{TD}$, $\lambda = 1 = \text{MC}$

Exam Success: Understand the bias-variance trade-off!

Deep Reinforcement Learning - Scaling RL

The Problem with Tabular Methods:

- Q-table stores $Q(s,a)$ for every state-action pair
- Number of states can be **enormous**
- Example: Atari games have 10^{50} states!

The Solution: Deep RL

- Use **neural networks** to approximate $Q(s,a)$
- $Q(s, a) \approx Q(s, a; \theta)$
- Neural network takes state as input, outputs Q-values

Why Neural Networks?

- 1 **Generalization**: Similar states get similar Q-values
- 2 **Compact representation**: No need to store all states
- 3 **Continuous spaces**: Handle continuous inputs

Exam Trap: Deep RL needed for large state spaces!

Deep Q-Network - The Breakthrough

What is DQN?

- Combines Q-learning with [deep neural networks](#)
- Developed by DeepMind (2015)
- Human-level performance on Atari games

Three Key Innovations:

- 1 [Experience Replay](#): Random sampling breaks correlation
- 2 [Target Network](#): Separate network for stable targets
- 3 [\$\epsilon\$ -Greedy](#): Exploration/exploitation balance

How It Works:

- 1 Interact with environment, store experiences
- 2 Sample random mini-batch from replay memory
- 3 Update policy network using TD target from target network
- 4 Periodically copy weights to target network

Exam Trap: DQN = Experience Replay + Target Network!

Experience Replay - Why It's Critical

The Problem:

- Consecutive experiences are **highly correlated**
- In games, consecutive frames are nearly identical
- Correlated data breaks i.i.d. assumption

The Solution:

- 1 Store experiences (s, a, r, s') in **replay memory**
- 2 Sample **random mini-batches** from memory
- 3 Breaks temporal correlation

Benefits:

- 1 **Stable learning**: Breaks correlations
- 2 **Efficient**: Reuses experiences
- 3 **Reduces variance**: Averaging over random samples

Replay Memory:

- Fixed size (e.g., 1,000,000)
- Oldest experiences discarded (FIFO)
- Random sampling

Exam Trap: Experience replay breaks temporal correlation!

Target Network - Stabilizing Learning

The Problem:

- Q-learning uses $R + \gamma \max_{a'} Q(s', a')$ as target
- But $Q(s', a')$ is the **same network** being updated!
- Creates "chasing the moving target" problem

The Solution:

- Use **separate network** for target
- Target network weights: θ^-
- Policy network weights: θ

How It Works:

- 1 Policy network: selects actions
- 2 Target network: computes TD target
- 3 Update: $L = (Q(s, a; \theta) - (R + \gamma \max_{a'} Q(s', a'; \theta^-)))^2$
- 4 Periodically copy: $\theta^- \leftarrow \theta$

Benefits:

- Stable targets
- More stable learning
- Helps convergence

Exam Trap: Target network updated periodically!

DQN Loss Function and Training

Loss Function (MSE):

$$L(\theta) = \mathbb{E}[(Q(s, a; \theta) - (R + \gamma \max_{a'} Q(s', a'; \theta^-)))^2]$$

Training Process:

- ❶ Sample random mini-batch from replay memory
- ❷ For each experience:
 - ❶ Compute target: $y = R + \gamma \max_{a'} Q(s', a'; \theta^-)$
 - ❷ Compute prediction: $Q(s, a; \theta)$
 - ❸ Compute loss: $(y - Q(s, a; \theta))^2$
- ❸ Average loss over mini-batch
- ❹ Backpropagate and update θ

Hyperparameters:

- Learning rate α : 0.001
- Mini-batch size: 32
- Replay memory: 1,000,000
- $\gamma = 0.99$

Exam Trap: DQN loss is MSE between policy and target!

DQN Training - Step-by-Step

Algorithm:

- ➊ Initialize:
 - Policy network $Q(s, a; \theta)$ with random weights
 - Target network $Q(s, a; \theta^-)$ with same weights
 - Replay memory D
- ➋ For each episode:
 - ➊ Initialize state s
 - ➋ For each step:
 - ➊ With probability ϵ : choose random action
 - ➋ Otherwise: $a = \arg \max_a Q(s, a; \theta)$
 - ➌ Take action, observe r, s'
 - ➍ Store (s, a, r, s') in D
 - ➎ Sample mini-batch from D
 - ➏ Compute loss and update θ
 - ➐ Periodically update $\theta^- \leftarrow \theta$
 - ➑ $s \leftarrow s'$

ϵ Decay:

- Start $\epsilon = 1.0$, decay to 0.01
- Linear decay over 1,000,000 steps

Exam Trap: Two networks: policy and target!

DQN Extensions - Making It Better

Double DQN:

- Uses target network for action selection too
- Reduces overestimation bias
- $y = R + \gamma Q(s', \arg \max_{a'} Q(s', a'; \theta); \theta^-)$

Dueling DQN:

- Separate value and advantage streams
- Better estimate of state values
- $Q(s, a) = V(s) + (A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'))$

Prioritized Experience Replay:

- Sample experiences with higher TD error more often
- Focuses on most "surprising" experiences
- Improves learning efficiency

Rainbow:

- Combines multiple DQN improvements
- State-of-the-art performance

Exam Trap: Double DQN and Dueling DQN are key extensions!

DQN Limitations

Limitations:

- 1 Discrete action spaces only
- 2 Computationally expensive
- 3 Sample inefficient
- 4 Can be unstable
- 5 Requires careful tuning

When to Use DQN:

- Large discrete action spaces
- High-dimensional state spaces (images)
- When you have computational resources
- When samples are plentiful

When NOT to Use DQN:

- Small state spaces (tabular better)
- Continuous action spaces (use policy methods)
- When samples are expensive

Exam Trap: DQN is only for discrete actions!

Policy-Based Methods - Learning Strategy Directly

Value-Based vs Policy-Based:

Value-Based	Policy-Based
Learn $Q(s,a)$	Learn $\pi(a s)$ directly
Derive policy from Q	Policy is the output
Q-learning, SARSA	Policy Gradient
Discrete actions only	Continuous actions possible

Policy Representation:

- Policy parameterized: $\pi_{\theta}(a|s)$
- θ = parameters (neural network weights)
- Output: probability distribution over actions

Why Policy Methods?

- 1 Continuous action spaces
- 2 Stochastic policies
- 3 Better convergence

Exam Trap: Policy methods for continuous actions!

REINFORCE - Monte Carlo Policy Gradient

Update Formula:

$$\theta \leftarrow \theta + \alpha \gamma^t G_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

Intuition:

- If G_t positive: increase probability of action
- If G_t negative: decrease probability of action
- Gradient points in direction to increase action probability

Advantages:

- Simple to implement
- Theoretically sound
- Works for continuous actions

Disadvantages:

- High variance (like MC)
- Requires complete episodes
- Slow convergence

Exam Trap: REINFORCE is MC-based with high variance!

Actor-Critic - Best of Both Worlds

What is Actor-Critic?

- Combines **value-based** and **policy-based**
- **Actor**: Learns the policy
- **Critic**: Learns the value function
- Critic evaluates actor's actions

How It Works:

- 1 Actor takes action based on current policy
- 2 Critic evaluates the action (TD error)
- 3 TD error: $\delta = R + \gamma V(s') - V(s)$
- 4 If $\delta > 0$: increase probability
- 5 If $\delta < 0$: decrease probability

Advantages:

- 1 **Lower variance** than policy gradient
- 2 **Can learn online** (TD-based)
- 3 **Continuous action spaces**

Exam Trap: Actor = policy; Critic = value!

Markov Decision Processes

What is an MDP?

- Mathematical framework for decision-making
- Defined by: States, Actions, Transitions, Rewards
- Assumes [Markov property](#)

MDP Components:

- 1 [States \(\$S\$ \)](#): Set of all states
- 2 [Actions \(\$A\$ \)](#): Set of all actions
- 3 [Transition Probability](#): $P(s' | s, a)$
- 4 [Reward](#): $R(s, a)$ or $R(s, a, s')$
- 5 [Discount Factor](#): γ

Markov Property:

$$P(S_{t+1} | S_t, S_{t-1}, \dots, S_1) = P(S_{t+1} | S_t)$$

- Future depends only on [current state](#)
- "Memoryless" property

Exam Trap: MDPs assume Markov property!

Bellman Equations - The Recursive Foundation

For a Fixed Policy:

$$V(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a) + \gamma V(s')]$$

Bellman Optimality:

$$V^*(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a) + \gamma V^*(s')]$$

For Q-Function:

$$Q(s, a) = \sum_{s'} P(s'|s, a) [R(s, a) + \gamma \max_{a'} Q(s', a')]$$

Key Insight:

- Value of state = immediate reward + discounted future value
- **Recursive** relationship
- Can be solved with dynamic programming

Exam Trap: Bellman equations are recursive!

Bellman Equations - Worked Example

Simple 2-State MDP:

- 2 states: $S1, S2$
- 2 actions: $A1, A2$ (deterministic)
- Rewards: $R(S1, A1, S2)=10, R(S2, A2, S1)=5$
- $\gamma = 0.9$

Bellman Equations:

- $V(S1) = 10 + \gamma V(S2)$
- $V(S2) = 5 + \gamma V(S1)$

Solve:

- $V(S1) = 10 + 0.9V(S2)$
- $V(S2) = 5 + 0.9V(S1)$
- Substitute: $V(S1) = 10 + 0.9(5 + 0.9V(S1))$
- $0.19V(S1) = 14.5$
- $V(S1) = 76.32$
- $V(S2) = 5 + 0.9(76.32) = 73.69$

Exam Trap: Bellman equations can be solved analytically!

RLHF - The Secret Behind Modern LLMs

What is RLHF?

- Uses [human feedback](#) as reward signal
- Humans rank or score model outputs
- RL algorithm maximizes human scores

Why It Matters:

- Key to ChatGPT and other LLMs
- Improves [instruction following](#)
- Enforces [policy constraints](#)
- Enhances [reasoning coherence](#)

How It Works:

- 1 Train a reward model on human preferences
- 2 Fine-tune policy using RL (PPO)
- 3 Generate responses and get feedback
- 4 Update policy to maximize reward

Exam Trap: RLHF is for LLM fine-tuning!

Model-Based vs Model-Free RL

Model-Based vs Model-Free RL

Model-Based RL:

- Uses **model** of environment
- Can plan ahead
- More sample efficient
- Risk: **Model bias**
- Example: AlphaZero

Model-Free RL:

- Learns directly from experience
- No model needed
- Less sample efficient
- No model bias
- Example: DQN, PPO

Comparison:

Model-Based	Model-Free
Uses model	No model
More sample efficient	Less sample efficient
Risk of model bias	No model bias
Planning possible	No planning

Exam Trap: Model-based = more efficient but risk of bias!

Reinforcement Learning - Complete Summary

Core Concepts:

- Agent learns through **trial and error**
- Goal: **Maximize cumulative reward**
- Policy: Strategy for choosing actions
- Value functions: $V(s)$ and $Q(s,a)$

Three Strategies (MAB):

- 1 Greedy (Pure Exploitation)
- 2 Random (Pure Exploration)
- 3 ϵ -Greedy (Balanced) - **Best**

Two Main Approaches:

- 1 **Monte Carlo (MC):**
 - Episode-end updates, actual returns
 - Unbiased, high variance, episodic only
- 2 **Temporal Difference (TD):**
 - Step-by-step updates, bootstrapping
 - Biased, low variance, episodic + continuous

Two Families:

- 1 **Value-Based:** Q-learning, SARSA, DQN
- 2 **Policy-Based:** REINFORCE, Actor-Critic

Exam Success: Master the exploration-exploitation trade-off!

Top 15 RL Exam Traps

- 1 RL output is a **policy**, not a prediction
 - 2 MAB has **ONLY ONE** state
 - 3 ϵ -greedy: random $< \epsilon \rightarrow$ **explore**
 - 4 $\epsilon_t = \beta^{t-1}$ (not β^t)
 - 5 MC updates **at episode end** using G_t
 - 6 TD updates **every step** using $R_{t+1} + \gamma V(s')$
 - 7 MC = **unbiased but high variance**
 - 8 TD = **biased but low variance**
 - 9 Q-learning is **off-policy**
 - 10 SARSA is **on-policy**
 - 11 α = learning rate, γ = discount factor, ϵ = exploration rate
 - 12 DQN uses **experience replay** + **target network**
 - 13 Policy methods handle **continuous action spaces**
 - 14 MC requires **episodic tasks**
 - 15 Bellman equations are **recursive**
- Remember:** RL is about **learning optimal behavior through experience!**